🌳Binary Tree Interview Cheat Sheet (JavaScript)

Target: 3–5 years Frontend (React / Angular / JS)

🐓Binary Tree Basics

- Each node has **at most 2 children** (left, right)
- **Root**: top node
- **Leaf**: node with no children
- **Height**: longest path from root to leaf

```
class Node {
  constructor(value) {
    this.value = value;
    this.left = null;
    this.right = null;
  }
}
```

🐕Sample Tree (Use for Dry Runs)

```
          10
         /
        5    20
       / \
      3   7     30
```

🦔DFS Traversals (MOST ASKED)

Inorder (Left → Root → Right)

```
function inorder(root) {
  if (!root) return;
  inorder(root.left);
  console.log(root.value);
  inorder(root.right);
}
```

Output: `3 5 7 10 20 30`

Preorder (Root → Left → Right)

```
function preorder(root) {
  if (!root) return;
  console.log(root.value);
  preorder(root.left);
  preorder(root.right);
}
```

Output: `10 5 3 7 20 30`

Postorder (Left → Right → Root)

```
function postorder(root) {
  if (!root) return;
  postorder(root.left);
  postorder(root.right);
  console.log(root.value);
}
```

Output: `3 7 5 30 20 10`

🐚 BFS (Level Order Traversal)

```
function levelOrder(root) {
  let queue = [root];
  while (queue.length) {
    let node = queue.shift();
    console.log(node.value);
    if (node.left) queue.push(node.left);
    if (node.right) queue.push(node.right);
  }
}
```

Output: `10 5 20 3 7 30`

🐜Height / Max Depth ⭐⭐⭐

```
function height(root) {
  if (!root) return 0;
  return 1 + Math.max(height(root.left), height(root.right));
}
```

Output: `3`

🐝Count Leaf Nodes

```
function countLeaf(root) {
  if (!root) return 0;
  if (!root.left && !root.right) return 1;
  return countLeaf(root.left) + countLeaf(root.right);
}
```

🐟Search in Binary Tree

```
function search(root, val) {
  if (!root) return false;
  if (root.value === val) return true;
  return search(root.left, val) || search(root.right, val);
}
```

🐠Validate Binary Search Tree (BST) ⭐⭐⭐⭐

```
function isBST(root, min = -Infinity, max = Infinity) {
  if (!root) return true;
  if (root.value <= min || root.value >= max) return false;
  return (
    isBST(root.left, min, root.value) &&
    isBST(root.right, root.value, max)
  );
}
```

🐡Invert / Mirror Binary Tree

```
function invertTree(root) {
  if (!root) return null;
  [root.left, root.right] = [root.right, root.left];
  invertTree(root.left);
  invertTree(root.right);
  return root;
}
```

🌈Lowest Common Ancestor (LCA) ⭐⭐⭐⭐

```
function LCA(root, p, q) {
  if (!root) return null;
  if (root.value === p || root.value === q) return root;

  let left = LCA(root.left, p, q);
  let right = LCA(root.right, p, q);

  if (left && right) return root;
  return left ? left : right;
}
```

🐐Complexity (MEMORIZE)

Operation	Time	Space
Traversal	O(n)	O(h)
BFS	O(n)	O(n)
Search (BT)	O(n)	O(h)
Search (BST)	O(log n)	O(h)

🌀Interview One-Liners

- **DFS** → Inorder, Preorder, Postorder
- **BFS** → Level order using Queue
- **BST faster** → eliminates half tree each step

- **Height** → `1 + max(left, right)`

🦜 Revision Priority (If Time Is Less)

1. Traversals
2. BFS
3. Height
4. BST Validation
5. LCA

Tip: Always explain **base case → recursion → return value** in interview.